

Inigo Quilez :: articles :: 2D axis aligned bounding boxes

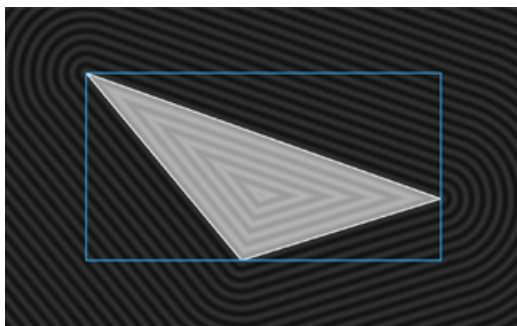


Intro

This is a list of 2D Axis Aligned Bounding Boxes (AABBs) for many of the SDF primitives in my [2D SDF functions](#). Computing bounding boxes is important for culling and pruning of primitives, which allows for faster rendering and physics of content made out of SDF shapes. For example, raymarching can be accelerated a big deal by only considering the primitives for which the bounding boxes overlap a tiled set of frustums. Computing the 3D boxes of an [extrusion](#) or [revolution](#) is trivial and should normally be based on the 2D bounding boxes below.

The parameters of all functions below match exactly those of the [corresponding 2D SDFs](#). The return value of all these functions is a `vec4`, where the `.xy` components contain the minimum (of bottom left) corner of the box, and the `.zw` components contain the maximum (or top right) corner of the box.

Note that all the primitives listed here come with a link to a realtime online demo in Shadertoy. In fact, this public playlist contains all the Shadertoy examples: <https://www.shadertoy.com/playlist/mcGBDK>, so don't miss that out.



Triangle

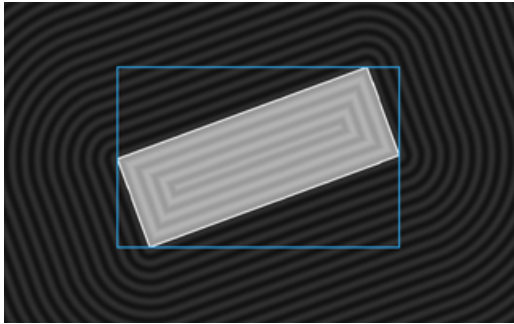
<https://www.shadertoy.com/view/tX2BDw>

```
vec4 aabbTriangle( in vec2 p0, in vec2 p1, in vec2 p2)
{
```

```

return vec4( min(p0,min(p1,p2)),
              max(p0,max(p1,p2)) );
}

```



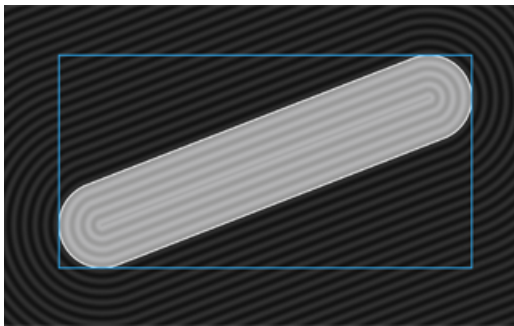
Oriented Box

<https://www.shadertoy.com/view/t32BWw>

```

vec4 aabbOrientedBox( in vec2 a, in vec2 b, in float r )
{
    vec2 v = r*abs(normalize(vec2(b.y-a.y,b.x-a.x)));
    return vec4(min(a,b)-v,max(a,b)+v);
}

```



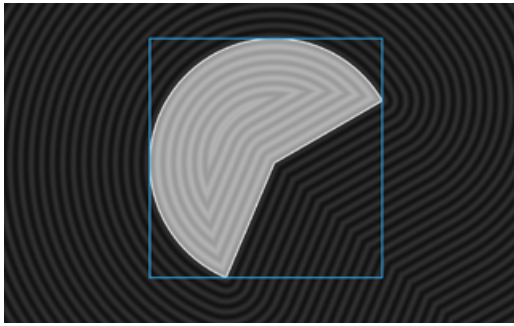
Segment

<https://www.shadertoy.com/view/332BDm>

```

vec4 aabbSegment( in vec2 a, in vec2 b, in float r )
{
    return vec4(min(a,b)-r,max(a,b)+r);
}

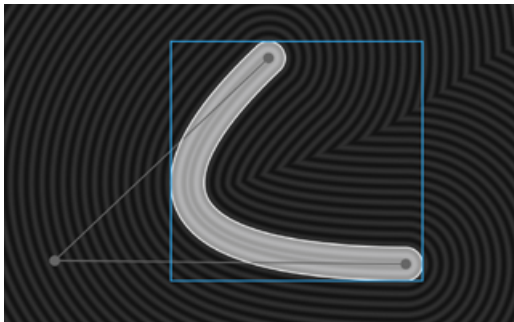
```



Pie

<https://www.shadertoy.com/view/t3ffDS>

```
vec4 boxPie( in vec2 c, in vec2 d, in float a, in float r )
{
    float si = sin(a);
    float co = cos(a);
    vec2 m = (d.xy)*co;
    vec2 n = abs(d.yx)*si;
    return c.xyxy + r*vec4(
        (d.x>-co) ? min(m.x-n.x,0.0) : -1.0,
        (d.y>-co) ? min(m.y-n.y,0.0) : -1.0,
        (d.x< co) ? max(m.x+n.x,0.0) :  1.0,
        (d.y< co) ? max(m.y+n.y,0.0) :  1.0 );
}
```



Quadratic Bezier

<https://www.shadertoy.com/view/lSyfWc>

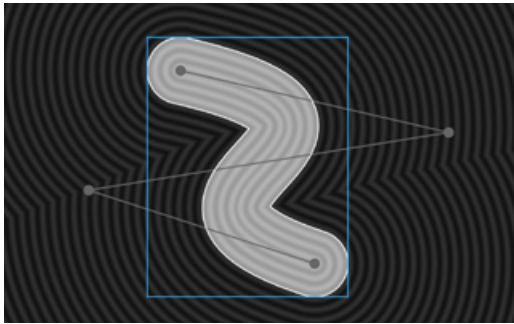
<https://iquilezles.org/articles/bezierbbox/>

```
vec4 aabbBezier( in vec2 p0, in vec2 p1, in vec2 p2 )
{
    vec2 a = p0-2.0*p1+p2;
    vec2 b = p1-p0;
    vec2 t = clamp(-b/a,0.0,1.0);
```

```

vec2 q = p0+t*(2.0*b+t*a);
return vec4(min(min(p0,p2),q),
            max(max(p0,p2),q));
}

```



Cubic Bezier

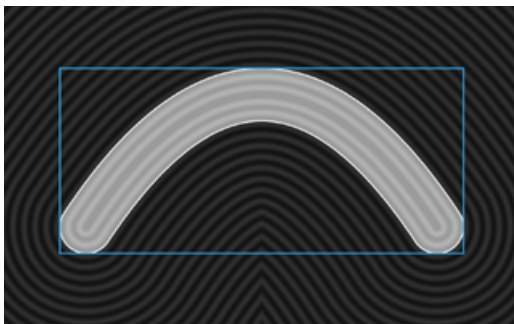
<https://www.shadertoy.com/view/XdVBWd>

<https://iquilezles.org/articles/bezierbbox/>

```

vec4 aabbBezier( in vec2 p0, in vec2 p1, in vec2 p2, in vec2 p3 )
{
    vec2 c = -p0+    p1;
    vec2 b = p0-2.0*p1+    p2;
    vec2 a = -p0+3.0*p1-3.0*p2+p3;
    vec2 g = sqrt(max(b*b-a*c,0.0));
    vec2 t1 = clamp((-b-g)/a,0.0,1.0);
    vec2 t2 = clamp((-b+g)/a,0.0,1.0);
    vec2 q1 = p0+t1*(3.0*c+t1*(3.0*b+t1*a));
    vec2 q2 = p0+t2*(3.0*c+t2*(3.0*b+t2*a));
    return vec4(min(min(p0,p3),min(q1,q2)),
                max(max(p0,p3),max(q1,q2)));
}

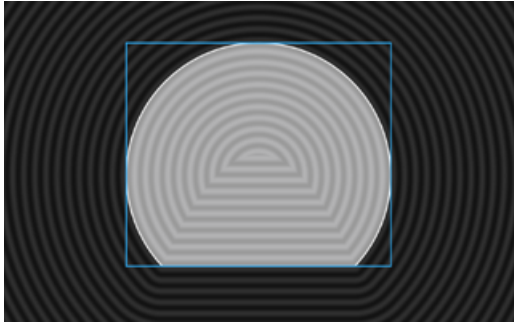
```



Parabola

<https://www.shadertoy.com/view/t3jBWw>

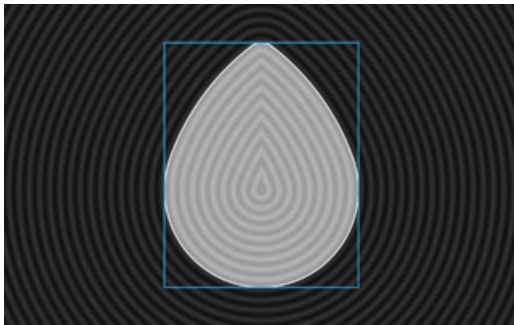
```
vec4 aabbParabola( in float w, in float h, in float r )
{
    return vec4(-w-r,min(h,0.0)-r,
                w+r,max(h,0.0)+r);
}
```



Cut Disk

<https://www.shadertoy.com/view/t3jfDw>

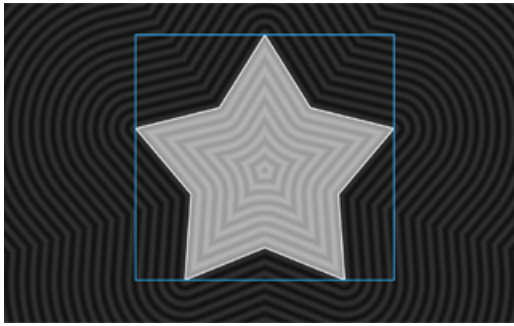
```
vec4 aabbCutDisk( in float r, in float h )
{
    float m = h>0.0 ? sqrt(r*r-h*h) : r;
    return vec4(-m,h,m,r);
}
```



Egg

<https://www.shadertoy.com/view/tXjfDw>

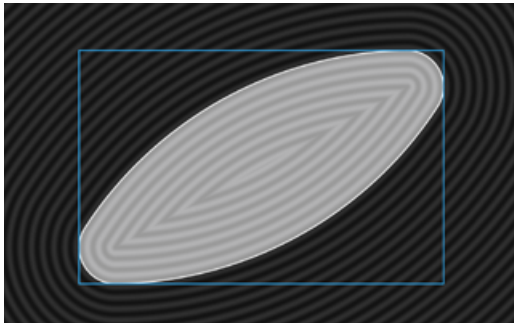
```
vec4 aabbEgg( in float he, in float ra, in float rb, in float bu )
{
    float wi = max(ra, rb);
    float r = 0.5*(he+ra+rb)/bu;
    float da = r - ra;
    float db = r - rb;
    float h = db*db - da*da;
    if( abs(h)<he*he )
    {
        float y = (h-he*he)/(2.0*he);
        wi = max(wi, r - sqrt(da*da-y*y));
    }
    return vec4(-wi, -ra, wi, he + rb);
}
```



Star

<https://www.shadertoy.com/view/tXjfWw>

```
vec4 boxStar( in float r, in int n, in float w)
{
    float an = 6.283185/float(n);
    vec2 kk = vec2( cos( round(float(n)/2.0)*an ),
                   sin( round(float(n)/4.0)*an ) );
    return r*vec4(-kk.y, kk.x, kk.y, 1.0);
}
```



Vesica Segment

<https://www.shadertoy.com/view/wccczH>

```
vec4 boxVesicaSegment( in vec2 a, in vec2 b, in float w )
{
    vec2 c = (b+a)*0.5;
    vec2 v = (b-a)*0.5;
    float v2 = dot(v,v);
    float d = 0.5*(v2-w*w)/w;
    float h = -v2/(d+w);
    vec2 p = abs(v.yx)*d/sqrt(v2);
    vec2 q = max(p-d-w,h);
    return vec4( min(min(a,b),c+q),
```

```
max(max(a,b),c-q) );
```

```
}
```

inigo quilez - learning computer graphics since 1994